STRINGFEST ANALYTICS

Prompt & Code Cheat Sheet

Every line of Python from tonight — copy-paste ready

Plum Solutions — Financial Modelling in Excel meetup · Presented by George Mount

Setup

In any Excel cell, type =PY(to switch that cell to Python. Press Ctrl+Enter to run. Reference an Excel Table with xl("tablename[#All]", headers=True). Start every session with the imports below.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

One data rule

Everything here uses fictional training data. In the office, keep real, identifiable financial or personal data out of any tool you would not put it in otherwise — and confirm your org's Python in Excel data policy.

Part 1 — Forecasting

Load and index the series

```
df = xl("revenue[#All]", headers=True)
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')
df.head()
```

Resample and smooth

```
quarterly = df.resample('QE').mean()
df['rolling'] = df['revenue'].rolling(window=12).mean()
```

The first 11 rolling values are NaN (shown as #NUM! in the grid). That is expected, not an error.

Decompose: trend + seasonal + residual

```
from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(df['revenue'], model='additive', period=12)
result.plot()
```

If the seasonal swings grow as the level rises, switch to model='multiplicative'.

Forecast with Holt-Winters

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
fit = ExponentialSmoothing(df['revenue'], trend='add',
                           seasonal='add', seasonal_periods=12).fit()
forecast = fit.forecast(12)
```

Evaluate against a naive baseline

```
train = df[df.index < '2024-01-01']
test  = df[df.index >= '2024-01-01']
fit = ExponentialSmoothing(train['revenue'], trend='add',
                           seasonal='add', seasonal_periods=12).fit()
fc = fit.forecast(len(test))
mae = (fc - test['revenue']).abs().mean()

# naive baseline: next month = last observed value
baseline = pd.Series(train['revenue'].iloc[-1], index=test.index)
baseline_mae = (baseline - test['revenue']).abs().mean()
f"HW MAE {mae:.0f}  vs  baseline MAE {baseline_mae:.0f}"
```

Match the method to the data

The data shows...	Use
Trend + stable seasonality	Holt-Winters (trend='add', seasonal='add')
Trend, no reliable seasonality	Holt's method (trend='add', seasonal=None)
Neither — fluctuates around a level	Simple exponential smoothing

Decompose first (try the bookings table). Let the seasonal panel tell you which method fits.

Part 2 — Simulation (Monte Carlo)

Generate random demand

```
np.random.seed(42)          # reproducible
N = 10000
demand = np.maximum(np.random.normal(1000, 200, N), 0)
```

Compute profit for every scenario

```
price, cost, inv = 50, 30, 1100
units   = np.minimum(demand, inv)   # =MIN(demand, inventory)
revenue = units * price
profit  = revenue - inv * cost
```

Summarize the distribution

```
p10, p90  = np.percentile(profit, [10, 90])
prob_loss = (profit < 0).mean()
summary = pd.DataFrame({'mean':[profit.mean()], 'median':[np.median(profit)],
                        'p10':[p10], 'p90':[p90], 'prob_loss':[prob_loss]})
summary
```

Answer a probability question

```
prob_high = (profit > 18000).mean()
f"Probability profit > $18,000: {prob_high*100:.1f}%"
```

Compare decisions

```
rows = []
for inv in [900, 1100, 1300]:
    p = np.minimum(demand, inv) * price - inv * cost
    rows.append({'inventory':inv, 'mean':round(p.mean()),
                 'p10':round(np.percentile(p,10)),
                 'prob_loss':(p<0).mean()})
pd.DataFrame(rows)
```

Want the whole toolkit?
These techniques and more — regression, text analytics, visualization — are in Python in Excel for Data Analytics (Packt), out by end of year. Get launch-day notice: stringfestanalytics.com/book